

PROGRAMMING LANGUAGES FOR DATA ANALYSIS

Python, R, SQL, and Rust Ecosystems

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- Statistical computing and modeling
- Database queries and data retrieval
- High-performance batch and stream processing



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- **Statistical computing and modeling**
- Database queries and data retrieval
- High-performance batch and stream processing



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- **Statistical computing and modeling**
- **Database queries and data retrieval**
- High-performance batch and stream processing



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- **Statistical computing and modeling**
- **Database queries and data retrieval**
- **High-performance batch and stream processing**



Language Selection Criteria

Performance Factors:

- Execution speed
- Memory efficiency
- Parallelization
- Scalability

Practicality Factors:

- Ecosystem maturity
- Learning curve
- Community support
- Integration capabilities



Language Selection Criteria

Performance Factors:

- Execution speed
- Memory efficiency
- Parallelization
- Scalability

Practicality Factors:

- Ecosystem maturity
- Learning curve
- Community support
- Integration capabilities



Outline

- 1 Overview
- 2 Programming Languages**
- 3 Language Comparison and Selection



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- Largest ecosystem: NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- Gentle learning curve: clear syntax, beginner-friendly



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- Gentle learning curve: clear syntax, beginner-friendly
- Interactive development: Jupyter notebooks for exploratory analysis
- Production-ready: from prototypes to deployment-grade systems



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve:** clear syntax, beginner-friendly
- **Interactive development:** Jupyter notebooks for exploratory analysis
- **Production-ready:** from prototypes to deployment-grade systems



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve:** clear syntax, beginner-friendly
- **Interactive development:** Jupyter notebooks for exploratory analysis
- **Production-ready:** from prototypes to deployment-grade systems



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem**: NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve**: clear syntax, beginner-friendly
- **Interactive development**: Jupyter notebooks for exploratory analysis
- **Production-ready**: from prototypes to deployment-grade systems



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with minimal code.



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with minimal code.



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with minimal code.



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with minimal code.



Python Core Libraries II

Statistical Learning

- **SciPy** (scipy): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (sklearn): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodels** (statsmodels): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (`scipy`): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (`sklearn`): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodel** (`statsmodels`): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (`scipy`): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (`sklearn`): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodels** (`statsmodels`): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (`scipy`): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (`sklearn`): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodels** (`statsmodels`): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- Statistical rigor: comprehensive hypothesis testing and model diagnostics
- Visualization excellence: publication-quality plots via `ggplot2`



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via `ggplot2`
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via `ggplot2`
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R Core Packages

Data Manipulation:

- dplyr: grammar of **data manipulation**
- tidyr: **reshaping** and pivoting
- data.table:
high-performance tables

Statistics & Visualization:

- ggplot2: layered graphics
- psych: descriptive statistics
- lme4: mixed-effects models
- caret: ML model training

Tidyverse Philosophy

The **Tidyverse** ecosystem (dplyr, ggplot2, tidyr, etc.) emphasizes **consistent APIs, tidy data principles**, and readable pipelines that flow naturally from left to right.



R Core Packages

Data Manipulation:

- dplyr: grammar of **data manipulation**
- tidyr: **reshaping** and pivoting
- data.table: **high-performance** tables

Statistics & Visualization:

- ggplot2: layered **graphics**
- psych: descriptive **statistics**
- lme4: mixed-effects models
- caret: **ML** model training

Tidyverse Philosophy

The **Tidyverse** ecosystem (dplyr, ggplot2, tidyr, etc.) emphasizes **consistent APIs**, **tidy data principles**, and readable pipelines that flow naturally from left to right.



R Core Packages

Data Manipulation:

- dplyr: grammar of **data manipulation**
- tidyr: **reshaping** and pivoting
- data.table: **high-performance** tables

Statistics & Visualization:

- ggplot2: layered **graphics**
- psych: descriptive **statistics**
- lme4: mixed-effects models
- caret: **ML** model training

Tidyverse Philosophy

The **Tidyverse** ecosystem (dplyr, ggplot2, tidyr, etc.) emphasizes **consistent APIs, tidy data principles**, and readable pipelines that flow naturally from left to right.



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying*, *filtering*, *joining*, and *aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- Efficient data access: indexes, query optimization, pushdown
- Set operations: JOINS, GROUP BY, window functions



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY**: grouping aggregation and post-aggregation filters
- **HAVING**: filtering rows using total results and logical expressions



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



Connecting Python/R to Databases

Python **Connectors:**

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- `pandas.read_SQL()`: direct to DataFrame

R **Connectors:**

- DBI: database interface
- `dplyr::tbl()`: lazy SQL queries
- `dbplyr`: translate dplyr to SQL
- RPostgres, RMySQL

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating **large datasets** close to the database. Bring **curated results** into Python/R for *analysis & visualization*.



Connecting Python/R to Databases

Python Connectors:

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- `pandas.read_SQL()`: direct to DataFrame

R Connectors:

- DBI: database interface
- `dplyr::tbl()`: lazy SQL queries
- dbplyr: translate dplyr to SQL
- RPostgres, RMySQL

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating **large datasets** close to the database. Bring **curated results** into Python/R for *analysis & visualization*.



Connecting Python/R to Databases

Python Connectors:

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- `pandas.read_SQL()`: direct to DataFrame

R Connectors:

- DBI: database interface
- `dplyr::tbl()`: lazy SQL queries
- dbplyr: translate dplyr to SQL
- RPostgres, RMySQL

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating **large datasets** close to the database. Bring **curated results** into Python/R for *analysis & visualization*.



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- Blazing speed: compile-time optimizations, zero-cost abstractions
- Safe parallelism: data races prevented at compile time
- Efficient, composable operators for real-time streaming
- Low-level control over hardware resources



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Language Comparison Matrix

Criterion	Python	R	SQL	Rust
Ease of Learning	High	Medium	High	Low
Ecosystem Size	Largest	Very Large	N/A	Growing
Performance	Medium	Medium	Excellent	Excellent
Data Manipulation	Excellent	Excellent	N/A	Good
Visualization	Good	Excellent	N/A	Basic
Statistical Tests	Very Good	Excellent	N/A	Limited
Parallelization	Good	Limited	Built-in	Excellent
Production Deployment	Excellent	Good	Standard	Very Good
Interactive Development	Excellent	Very Good	Limited	Poor
ML / DL Support	Dominant	Good	None	Growing
Community Size	Largest	Large	Mature	Growing



When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, **safe concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, safe **concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, **safe concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

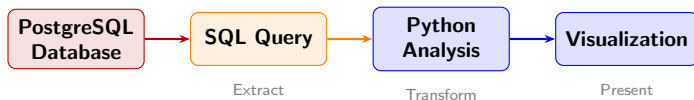
SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, safe **concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

The Modern Data Stack



- **Layer 1 (SQL):** Query database, filter, aggregate at source
- **Layer 2 (Python/R):** Load curated data, explore, model, validate
- **Layer 3 (Visualization):** Create dashboards, reports, and presentations



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

- ❶ **SQL:** Extract monthly sales by region from data warehouse
 - Filter to last 12 months
 - Group by region, product category
 - Calculate totals and growth rates
- ❷ **Python:** Load results, perform statistical analysis
 - Detect outliers and anomalies
 - Fit predictive model (scikit-learn)
 - Calculate correlations
- ❸ **R (optional):** Create publication-quality visualization
 - ggplot2 for advanced graphics
 - Statistical annotations



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

- ❶ **SQL:** Extract monthly sales by region from data warehouse
 - Filter to last 12 months
 - Group by region, product category
 - Calculate totals and growth rates
- ❷ **Python:** Load results, perform statistical analysis
 - Detect outliers and anomalies
 - Fit predictive model (scikit-learn)
 - Calculate correlations
- ❸ **R (optional):** Create publication-quality visualization
 - ggplot2 for advanced graphics
 - Statistical annotations



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

- ❶ **SQL:** Extract monthly sales by region from data warehouse
 - Filter to last 12 months
 - Group by region, product category
 - Calculate totals and growth rates
- ❷ **Python:** Load results, perform statistical analysis
 - Detect outliers and anomalies
 - Fit predictive model (scikit-learn)
 - Calculate correlations
- ❸ **R (optional):** Create publication-quality visualization
 - ggplot2 for advanced graphics
 - Statistical annotations



Best Practices for Multi-Language Work

- **Separation of concerns:** Each language handles what **it does best**
- **Data formats:** Use **Parquet** or **CSV** as interchange format
- **Version control:** **Track** code and data pipelines in **Git**
- **Reproducibility:** **Document** dependencies and environment **setup**
- **Profiling:** **Measure performance**, identify bottlenecks
- **Testing:** **Validate** transformations at each stage



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

